Autonomous Robotics & Reinforcement Learning: The Vision-Based Navigator

Project Assignment Guide of LSI UC

1. Introduction and Context

Welcome to your final challenge in Laboratory of Intelligent Systems. Your task is to develop a Reinforcement Learning (RL) agent capable of navigating a differential-drive robot (the e-puck) through an arena to locate and reach a specific target: **The Red Ball**.

You have been provided with a **Baseline Code** that includes a single controller that is also a supervisor (`epuck_supervisor.py`) and a dual controller composed of a regular e-puck controller (`epuck_controller.py`) that communicates through an Emitter and a Receiver with a supervisor (`rl_supervisor_env.py`). These allows you to select one of the approaches to train a reinforcement learning agent. The first is to be used with the world `empty_single_controller.wbt` and the second(s) with the `empty_lab_dual_controller.wbt`. Finally, the agents in both cases are trained and executed with the `rl_agent.py`, located in the rl directory. **Remember that your agent will be tested as a simple controller, without supervisor permissions**. This code features two fully functioning Gym Environments interacting with the Webots simulator. However, the current AI brain is effectively a "blank canvas". It operates on a **Sparse Reward System**—it only receives points when it accidentally hits the ball (+100) and loses points if it crashes head-on into a wall (-10).

If you run the baseline, you will notice the robot struggles to learn. It wanders aimlessly, gets stuck, and relies entirely on luck. **Your job is to fix this.**

2. Project Objectives

Your mission is to upgrade the robot's intelligence from a random wanderer to a robust, autonomous navigator. To do this, you must complete the following core objectives:

1. **Implement Reward Shaping:** Design a continuous reward function that guides the robot smoothly toward the goal without relying solely on the final jackpot, preventing looping, getting stuck, or engaging in "reward hacking" (e.g., finding a way to farm points without completing the maze).
2. **Achieve Generalization:** Your agent must not just memorize one specific arena. It must be capable of reaching the red ball in **modified environments** with different wall layouts, obstacle placements, and starting positions.

3. The Ultimate Challenge: Vision-Supported Navigation

The baseline code utilizes distance sensors (IR proximity), target coordinates, and angle calculations in its observation space, to simplify initial testing and understanding of RL mechanics.

CRITICAL CONSTRAINT: For your final evaluation, in addition to the proximity sensors (`ps0` to `ps7`), your agent must use the camera as well.

Your final model must navigate using the robot's onboard Video Camera and the proximity sensors to assist in obstacle and wall detection. Just like a human driver or a Tesla on Autopilot, your robot must look at the world, process the pixels, understand depth/obstacles implicitly, spot the red ball, and drive towards it.

Technical Implications of this Constraint:

- You will need to enable the camera device in the Webots environment (`self.camera = self.robot.getDevice('camera')`).
- Your `observation_space` will change from a 1D array (Box of shape 10) to a 3D image matrix (e.g., `Box(low=0, high=255, shape=(HEIGHT, WIDTH, 3), dtype=np.uint8)`). Remember that if you're using the proximity sensors, the observation space will need to be adjusted accordingly.
- You must transition from using an MLP (Multi-Layer Perceptron) policy to a **CNN (Convolutional Neural Network) Policy** (`CnnPolicy` in Stable Baselines3) to process the video feed.

4. Recommended Development Phases

To succeed, we highly recommend dividing your work into three distinct phases:

Phase 1: Mastering the Mathematics

- Keep the baseline sensors (proximity, distance, angles).
- Focus entirely on **Reward Shaping**. Experiment with giving the robot points for reducing its distance to the target, penalizing it for sharp, unnecessary turns, and heavily penalizing wall collisions.
- Prove to yourself that you can make the robot consistently solve the maze using numeric data.

Phase 2: The Vision Transition

- Implement the camera feed. You may want to downscale the image or convert it to grayscale/HSV to speed up the training process.
- *Hint:* Extracting a smaller resolution image (e.g., 64x64 pixels) will make your CNN train exponentially faster than using a high-definition feed.
- Change your SB3 algorithm to use `CnnPolicy`.

Phase 3: Generalization & Robustness

- Once your vision-based robot can find the ball in the standard arena, modify the Webots world. Move the walls, place boxes in the way, and change the lighting slightly.

- Train your agent across multiple randomized scenarios so it learns to recognize the *concept* of an obstacle and the *concept* of the red ball, rather than memorizing a specific path.

5. Evaluation & Grading Criteria

Your final grade will be determined by three components: Agent Performance, a Technical Report, and an Oral Presentation.

A. Agent Performance in Unseen Scenarios (40%)

On evaluation day, your saved model (`.zip` file) and your environment script will be tested live on the instructor's computer.

- **The Test:** We will place your robot in a completely new, unseen arena layout, and it will be restarted 3 times. For each restart the layout may change!
- **The Metric:** The robot must visually locate and navigate to the red ball within a given time limit without getting perpetually stuck or crashing uncontrollably.

B. Technical Report (30%)

Submit a concise IEEE-format paper (max 6 pages) documenting your engineering process. The report must include:

- **Abstract:** A summary of your approach and results.
- **Reward Engineering:** A detailed mathematical explanation of your final reward function. Why did you choose those specific values? What earlier versions failed, and why?
- **Observation Space Architecture:** How did you process the camera image? Did you apply any filters (e.g., cropping, color masking) before feeding it to the CNN?
- **Training Metrics:** Include graphs (e.g., from TensorBoard) showing your episodic reward accumulation and episode lengths over time.
- **Discussion of Generalization:** Explain why your agent succeeds (or fails) in different environments.

C. Oral Presentation & Defense (30%)

Your team will present your project to the class.

- **Duration:** 10 minutes presentation + 5 minutes Q&A.
- **Content:** Show a live demonstration or pre-recorded videos of your agent training and succeeding.
- **Defense:** Be prepared to answer technical questions about Reinforcement Learning parameters (e.g., learning rate, buffer size, exploration vs. exploitation) and your specific implementation choices.

D. Team Definition

You will have to work in teams! For that you must join one or two colleagues, so that the team size will be within a minimum of 2 and a maximum of 3 elements.

6. Tips for Success

- **Train Smart, Not Hard:** Training CNNs takes significantly more computational power than MLPs. Ensure your reward function is rock-solid using standard variables (Phase 1) before you waste 10 hours training a camera-based model on a broken reward logic.
- **Frame Stacking:** Consider looking into "Frame Stacking" wrappers in Gymnasium. Giving the network the last 3 or 4 frames instead of just 1 frame helps the AI understand *velocity* and *direction of movement*.
- **Beware of Reward Hacking:** If you give the robot +1 point for every step it takes away from a wall, it might just spin in circles in the middle of the room forever to collect infinite points. Always tie your highest rewards to actual progress toward the goal!

Good luck. Let the training begin!